

Table des matières

Exercices 08 — Données semi-structurées.....	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	4
Mises en situation	4

Exercices 08 — Données semi-structurées

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez trois heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué [OLTP] se traite dans PostgreSQL sur Adventureworks (schémas sales, production, person). Un énoncé marqué [OLAP] se traite dans DuckDB sur adventureworksdw.duckdb, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé est dans `Corrige_08.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Définissez « semi-structuré » en une phrase, sans employer le mot JSON.
2. Donnez trois sources qui produisent naturellement de la donnée semi-structurée, et dites ce qu'elles ont en commun.
3. json contre jsonb sur PostgreSQL. Citez trois différences observables, et dites lequel vous écrivez par défaut.
4. Dans quel cas précis json se justifie-t-il malgré tout ?
5. Combien de types JSON l'entrepôt offre-t-il, et que cela change-t-il par rapport au choix qu'impose PostgreSQL ?
6. Que rendent `jsonb_build_object('a', 1, 'b', NULL)` et `JSON_OBJECT('a', 1, 'b', NULL)` ? D'autres moteurs font l'inverse : dites lequel de leurs comportements est le plus dangereux, et pourquoi.

7. -> et ->>. Que rend chacun, et quelle est la règle de choix entre les deux ?
 8. Sur l'entrepôt, pourquoi `WHERE charge -> 'pays' LIKE 'Can%'` ne renvoie-t-il aucune ligne, et comment corrige-t-on ?
 9. Le mode de défaillance de la question 8 ressemble à celui d'un piège du Cours 07. Lequel, et qu'ont-ils en commun ?
 10. Une extraction rend-elle jamais un nombre ? Qu'arrive-t-il si on trie sans convertir ?
 11. CAST contre la conversion tolérante. Quand préférez-vous la seconde, que faites-vous ensuite, et comment s'écrit-elle sur chacun des deux terrains ? Dites enfin ce que l'écriture de PostgreSQL distingue et que celle de l'entrepôt confond.
 12. Aplatir un tableau. Nommez la fonction de chaque moteur, et dites de quel outil du Cours 07 elles ont besoin toutes les deux.
 13. `JSON_EACH` rend une colonne en plus de la valeur. Nommez-la, dites ce qu'elle contient sur un tableau, ce qu'elle contient sur un objet, et à quoi elle sert.
 14. Sur PostgreSQL, comment récupère-t-on la position d'un élément dans le tableau aplati ? Donnez les deux écritures, et dites laquelle type ses colonnes au passage.
 15. Qu'advient-il d'un document dont le tableau est vide quand on l'aplatit, sur chacun des deux terrains ? Quel outil du Cours 07 permet de le garder ?
 16. La forme du fichier. Un fichier qui contient `[ {...}, {...} ]` se lit de trois façons sur l'entrepôt. Dites laquelle donne le bon compte, et laquelle est la plus dangereuse des deux autres.
 17. Pourquoi charge-t-on un fichier JSON en brut, dans une colonne unique, avant de le typer ? Donnez l'argument en une phrase.
1. L'index GIN sert trois opérateurs. Citez-les et dites ce que chacun teste.
 2. Pourquoi un index GIN n'accélère-t-il pas `document ->> 'pays' = 'Canada'` ? (avancé)
 3. Citez les trois situations où stocker du JSON est une mauvaise décision, et donnez la règle qui les résume.
-

Questions pratiques

P1. [OLTP] Fabriquez un document par client contenant son identifiant et le tableau des numéros de ses commandes. Une seule requête, sans table intermédiaire.

P2. [OLAP] Reprenez la table `LAB008_BRUT` du laboratoire. Écrivez la requête qui compte, pour chaque client, le nombre de commandes et le montant total, en aplatissant le tableau des lignes. Attention aux conversions.

- P3. [OLTP] Sur `labo08.commande`, listez les dix commandes dont la première ligne porte le prix unitaire le plus élevé. Écrivez d'abord la version fautive sans conversion, notez le premier résultat, puis la version correcte, et dites en une phrase pourquoi elles diffèrent.
- P4. Écrivez, sur le terrain de votre choix, la requête qui trouve les documents dont le tableau `lignes` est vide ou absent. Faites-la deux fois : une fois en aplatisant, une fois sans, et dites laquelle vous gardez.
- P5. [OLTP] Créez un index GIN sur `labo08.commande`, puis écrivez deux requêtes équivalentes du point de vue du résultat, dont une seule emploie l'index. Prouvez-le avec `EXPLAIN ANALYZE`.
- P6. [OLAP] Écrivez la requête qui, pour chaque commande, renvoie le produit de la ligne au montant le plus élevé. Vous aurez besoin d'aplatir puis de classer : le Cours 07 vous a donné trois façons de faire un top N par groupe. (avancé)
- P7. [OLAP] Un fournisseur vous livre un fichier qui contient `[{...},{...},{...}]`. Écrivez la lecture qui produit une ligne par élément, puis la vue qui expose trois colonnes typées à partir de la colonne brute.
- P8. [OLTP] Une colonne `donnees_jsonb` d'une table applicative contient parfois `{"montant": "n/a"}`. Écrivez la requête qui calcule la somme des montants valides sans que la requête tombe, et qui renvoie aussi le nombre de valeurs écartées. (avancé)
-

Diagnostics

E1. Une analyste cherche la commande SO43697 sur l'entrepôt :

```
SELECT COUNT(*) FROM commande_brute WHERE (charge -> 'commande')::VARCHAR = 'SO43697';
```

Elle renvoie zéro alors que la commande existe. Expliquez et corrigez. Puis dites pourquoi la même écriture, appliquée au client, (`charge -> 'client'`)::VARCHAR = '11000', aurait trouvé ses lignes : c'est cette réussite-là qui rend le piège dangereux.

E2. Un rapport de tri sur PostgreSQL :

```
SELECT document ->> 'commande', document ->> 'montant'
FROM commande
ORDER BY 2 DESC
LIMIT 10;
```

L'auteur affirme afficher les dix plus gros montants. Expliquez pourquoi c'est faux, et corrigez.

E3. Un aplatissement qui perd des lignes :

```
SELECT c.charge ->> 'commande' AS commande, (l.value ->> 'produit')::INTEGER AS produit
FROM commande_brute AS c, JSON_EACH(c.charge -> 'lignes') AS l;
```

Le compte des commandes distinctes est inférieur au nombre de lignes de `commande_brute`. Donnez la cause et la correction.

E4. Une lecture de fichier sur l'entrepôt :

```
CREATE OR REPLACE TABLE brut AS
SELECT json AS charge FROM READ_JSON_OBJECTS('livraison.json');
SELECT COUNT(*) FROM brut;
```

Le fichier contenait quarante mille objets, la table en contient trois. Expliquez et corrigez.

E5. Une extraction sur PostgreSQL :

```
SELECT jsonb_array_elements(document ->> 'lignes') FROM commande;
```

La requête échoue. Expliquez l'erreur et corrigez-la. (avancé)

E6. Un architecte a modélisé une table de commandes ainsi : `id`, `date`, `client_id`, et une colonne `donnees jsonb` qui contient le montant, le statut, l'adresse de livraison et le mode de paiement. Ces quatre informations existent dans toutes les lignes depuis trois ans. Quatre défauts concrets découlent de ce choix. Nommez-les.

Questions à développer

- A1. La théorie affirme que le JSON est bon pour ce qui varie et mauvais pour ce qui est stable. Défendez cette règle en un paragraphe, puis décrivez le motif de conception qui permet de tenir les deux bouts dans un entrepôt.
- A2. On vous propose de typer les documents au chargement, pour éviter une conversion à chaque lecture. L'argument est réel. Répondez-lui en un paragraphe, en reconnaissant ce qu'il a de juste.
- A3. Comparez la façon dont chacun des deux terrains rend le semi-structuré performant. Votre réponse doit expliquer pourquoi l'un offre un index et l'autre non, dire ce que les grands entrepôts automatisent en plus, et nommer la parade qui vaut sur tous les moteurs.
- A4. Les trois fautes du module sont silencieuses : chemin inexistant, comparaison sans conversion, tri non typé. Proposez trois pratiques concrètes qui les rendraient visibles avant la production, et dites laquelle vous mettriez en place en premier. (avancé)
-

Mises en situation

M1. Plateforme de covoiturage Chemin-Vert, l'ingestion des trajets. La plateforme reçoit d'une application mobile un message par trajet terminé. Le message porte toujours un identifiant, un horodatage, un conducteur, un passager et un prix, plus un nombre variable de champs selon la version de l'application : un détail des péages, une note, un motif d'annulation, des étapes intermédiaires. Trois versions de l'application coexistent en production.

Concevez le chargement et l'exposition de cette donnée. Dites ce que vous stockez, sous quelle forme, ce que vous exposez en colonnes et ce que vous laissez en document. Précisez ce qui se passe quand la version 4 de l'application ajoute un champ, et pourquoi votre conception le supporte.

M2. Cabinet d'assurance Rive-Nord, le champ fourre-tout. Vous héritez d'une table *sinistre* de quatre millions de lignes. Elle a six colonnes propres et une colonne *complement_jsonb* qui contient, selon les lignes, entre deux et quarante clés. Personne ne sait ce qu'elle contient. Les rapports mensuels mettent quarante minutes à s'exécuter.

Proposez une démarche en trois temps : comprendre, mesurer, corriger. Pour le premier temps, dites précisément quelle requête vous écririez pour inventorier les clés et leur fréquence. Pour le troisième, dites quel critère décide qu'une clé mérite de devenir une colonne. Précisez enfin ce que vous ne feriez pas, et pourquoi.

M3. Courtier en énergie Sagamie, le flux de cotations. Le courtier reçoit d'un fournisseur un flux de cotations horaires, en JSON, environ deux millions de messages par mois. Chaque message porte un horodatage, un marché, un prix, et un bloc de métadonnées dont la forme a changé trois fois en deux ans. L'équipe veut un tableau de bord des prix moyens par marché et par jour, et l'audit exige de pouvoir reproduire n'importe quel message reçu, à l'octet près, pendant sept ans.

Ces deux exigences tirent dans des directions opposées. Concevez l'architecture de stockage qui les satisfait toutes les deux. Dites où vous mettez le brut, où vous mettez l'exploité, et comment vous garantissez que le second se régénère depuis le premier. Nommez enfin la contrainte de l'énoncé qui interdirait d'employer *jsonb* si l'on était sur PostgreSQL, et pourquoi. (avancé)